软 件 工 程

实验三：软件实现与构建

校园二手交易系统的编码实现、静态检查与仓库托管

课程名称 软件工程

项目名称 校园二手交易系统（CampusTrade）

学 号 241220107

姓 名 黄 毅

学 院 计算机科学与技术学院

专 业 计算机科学与技术

任课教师 王 豫

完成日期 2026 年 11 月

依据实验一 19 条需求与实验二 6 张 UML 图实现
Python 3 共 22 个源文件、3311 行（有效代码 2761 行），73 个单元测试
全部改动见 git 提交记录，远程仓库 APY-tech/Software-Engineering

目录

1 实现概述 1

1.1 实现目标与技术选型 1

1.2 总体结构 1

1.3 五个关键实现决定 2

1.4 与实验二组件的对应 2

2 从 UML 图到代码 3

2.1 转换规则 3

2.2 类图的落地 3

2.3 对实验二类图的三处修订 6

2.4 状态图的落地 6

2.5 时序图的落地 8

2.6 代码规模 9

3 大模型辅助开发 11

3.1 使用方式 11

3.2 大模型实际承担的工作 12

3.3 模型生成不符合预期的四个案例 12

3.4 我的判断 13

4 代码风格与静态检查 13

4.1 检查工具与配置 13

4.2 初稿的检查结果 13

4.3 两类典型修复 15

4.4 复查结果 15

5 编译与运行结果 16

5.1 运行环境 16

5.2 编译检查 16

5.3 单元测试 16

5.4 端到端演示 17

5.5 交互式运行 19

6 Git 远程代码管理 **20**

6.1 本地仓库与远程仓库 20

6.2 本次实验的提交 21

6.3 提交粒度的约定 21

6.4 入库范围 22

7 与设计的差异、未覆盖项与反思 **22**

7.1 与实验二设计的五处差异 22

7.2 未覆盖或简化的部分 22

7.3 反思 23

1 实现概述

1.1 实现目标与技术选型

本次实验要把实验二的设计变成可编译、可运行、可测试的软件：8 个组件、10 个领域类、7 个商品状态、19 条需求（15 条功能、3 条非功能、1 条待确认）。语言选 **Python 3.13**，只使用标准库，运行时不需要安装任何第三方包；界面采用**命令行菜单**，没有做图形界面。

选型理由有三条。第一，本系统的难点不在界面而在规则：同一商品至多一条有效预约、审核期间商品仍可被预约、已发布商品改内容要” 前台显示旧信息、后台审新内容”。这些规则的组合爆炸靠人工枚举容易漏，用 Python 写测试验证最省力。第二，后续实验要对程序做静态分析与测试，Python 生态里的 `pylint`、`pytest` 可以直接对被测代码做度量，不需要额外搭桩。第三，只用标准库可以保证在机房机器和自备机器上结果一致，避免环境差异带来的” 在我这里能跑”。

不做图形界面是权衡后的决定：实验二已给出 10 张界面稿，但本次的重点是规则正确性，控制台界面用 `Shell` 的 30 个菜单项把全部用例走通，界面层与业务层完全分离，将来换成 Tkinter 或 Web 只需替换 `campustrade/cli` 目录，服务层一行不改。代价是演示时看不到视觉反馈，这一点在第 7 节作为不足记录。

1.2 总体结构

代码分五层，依赖方向单向向下，界面层不允许直接碰仓储（表 1-1）。

表 1-1 代码分层与职责（行数含空行与注释）

层	文件	职责	行数
领域模型	`models.py`、`errors.py`、`state.py`	实体、枚举、值对象、异常层次、状态迁移表	513
基础设施	`storage.py`、`security.py`	内存表 + JSON 快照、口令散列与会话令牌	305
服务层	`services/`（9 个文件）	认证、商品、审核、交易、举报、统计、日志与通知	1343
界面层	`cli/shell.py`、`cli/tables.py`	菜单、输入校验、中文宽度对齐的表格	602
装配与入口	`context.py`、`seed.py`、`demo.py`、`run.py`	依赖装配、演示数据、端到端脚本、参数解析	548
合计	22 个源文件	——	3311

装配集中在 `context.py` 的 `build_application()`：先建仓储与日志，再建认证，最后建六个业务服务。依赖顺序写在一处，服务之间就不会互相 `import` 造成循环，测试里也能一行拿到整套服务。

1.3 五个关键实现决定

1. **状态只有一处可写**。商品状态不对外暴露 setter，所有迁移都走 `apply_status()`，它先查迁移表再赋值并递增版本号。7 个状态之间的 9 条合法迁移与实验二状态图一一对应，非法迁移抛 `Conflict`，错误信息里带当前状态和版本。
2. **权限判断跟着服务走，不跟着界面走**。每个面向界面的方法第一个参数都是会话令牌，方法内部换成 `User` 再判断角色与归属。界面层因此不可能”忘记”校验，测试也能直接拿不同角色的令牌调同一个方法。
3. **持久化用整库快照 + 原子替换**。写临时文件后 `os.replace()`，即使进程中途被杀也不会留下半截 JSON。课程作业的数据量在千条级，这个方案比引入数据库省事，也便于人工查看数据。
4. **并发只在必要处加锁**。仓储持有一把可重入锁，`TradeService.reserve()` 与 `cancel/confirm` 在临界区内完成”查重—校验—写入”，其余操作不加锁。锁的范围就是”同一商品至多一条有效预约”这条不变式需要保护的范围。
5. **异常分层**。`DomainError` 之下分五类，各管一件事：入参不合法 `ValidationError`、未登录或过期 `AuthError`、越权 `PermissionDenied`、目标不存在 `NotFound`、与当前状态冲突 `Conflict`。界面层只在一个地方捕获并打印，服务层不返回错误码、不打印日志。

1.4 与实验二组件的对应

实验二把系统划成 8 个模块（M1–M8），实现时按依赖方向落成 7 个服务加 1 个横切模块（表 1-2）。分量基本对齐，只有两处合并：M4 查询并入商品服务（查询的条件就是商品属性，拆开会出现两个服务同时读 `Item`）、M8 公共支撑拆成 `storage` 与 `journal` 两部分（一个管持久化、一个管日志与通知）。

表 1-2 实验二组件到实验三模块的映射

组件	实验二职责	实现位置	说明
M1	用户与权限	`services/auth.py`	注册、登录、会话过期、角色分配
M2	商品管理	`services/item.py`	草稿、修改、送审、下架、分类维护
M3	审核	`services/audit.py`	通过/驳回、审核记录、待审内容原子替换
M4	查询	并入 M2	按关键词、分类、价格、排序过滤
M5	交易	`services/trade.py`	预约、取消、确认、完成、超时自动完成
M6	举报治理	`services/moderation.py`	举报去重、下架/驳回处置
M7	日志与统计	`services/journal.py`、`stats.py`	日志只追加、两套统计口径
M8	公共支撑	`storage.py`、`journal.py`	快照持久化、通知

2 从 UML 图到代码

2.1 转换规则

UML 里的每种元素在代码里只对应一种写法，规则先定下来，后面 10 个类就是重复劳动，不会出现” 这个类用字典、那个类用元组” 的随意性（表 2-1）。

表 2-1 UML 元素到 Python 构造的转换规则

UML 元素	代码构造	处理要点
类（实体）	`@dataclass`	字段名、类型照抄；主键统一暴露 `ident` 属性，仓储据此泛型存取
枚举	`class X(str, Enum)`	继承 `str` 便于 JSON 落盘与命令行按名字解析
关联（1—N）	外键字段 + 仓储查询	不在实体里挂对象引用，避免循环引用与反序列化困难
组合（Item—ItemRevision）	独立表 + 按 `itemId` 查	待审内容能被单独丢弃，符合” 驳回即丢” 的语义
操作（方法）	服务层方法	见 2.3 的修订说明
状态图	迁移表 + `apply_status`	每条箭头一个表项，非法迁移抛异常
时序图	一个方法体 + 锁	消息顺序即语句顺序，`alt` 分支即 `if/else`

2.2 类图的落地

领域模型（图 2-1）是照实验二类图逐类翻译的：`User`、`Session`、`Item`、`ItemRevision`、`Reservation`、`Report`、`AuditRecord`、`Category`、`OperationLog`、`Notification` 十个类，加上 `Role`、`ItemStatus`、`ReservationStatus`、`ReportState`、`AuditResult` 五个枚举，全部落在 `models.py` 一个文件里（327 行）。实体只保留” 看自己” 的判断，例如 `Item.is_public()` 判断能否出现在公开查询里、`Item.has_published_version` 判断驳回后是回草稿还是继续展示旧信息。

服务层与仓储（图 2-2）是实验二类图里没有的，属于实现期新增的结构：`ServiceBase` 统一持有仓储、认证、日志三个依赖，六个业务服务继承它；`Store` 是仓储门面，`Table` 是泛型内存表，二者是组合关系。

每个类在代码里的位置与关键方法见表 2-2。

表 2-2　类与代码位置的对应（行号为该文件起始行）

类 / 枚举	文件	关键实现	起始行
User、Session、Role	`models.py` + `services/auth.py`	口令散列、会话过期判定	75 / 39
Item、ItemStatus	`models.py` + `services/item.py`	草稿、修改、送审、下架、公开查询	132 / 99
ItemRevision	`models.py` + `item.py`	`_stage_revision` 暂存待审内容	168 / 279
Category	`models.py` + `item.py`	分类增删改，被引用则拒绝删除	118 / 44
Reservation	`models.py` + `services/trade.py`	幂等预约、取消、确认、完成	205 / 32
Report、ReportState	`models.py` + `moderation.py`	重复举报返回原记录、处置复用下架	227 / 30
AuditRecord、AuditResult	`models.py` + `services/audit.py`	审核记录只追加、待审内容原子替换	187 / 52
OperationLog、Notification	`models.py` + `services/journal.py`	日志只追加、通知收发	247 / 24
状态迁移表	`state.py`	`can_transition` / `apply_status`	24 / 47

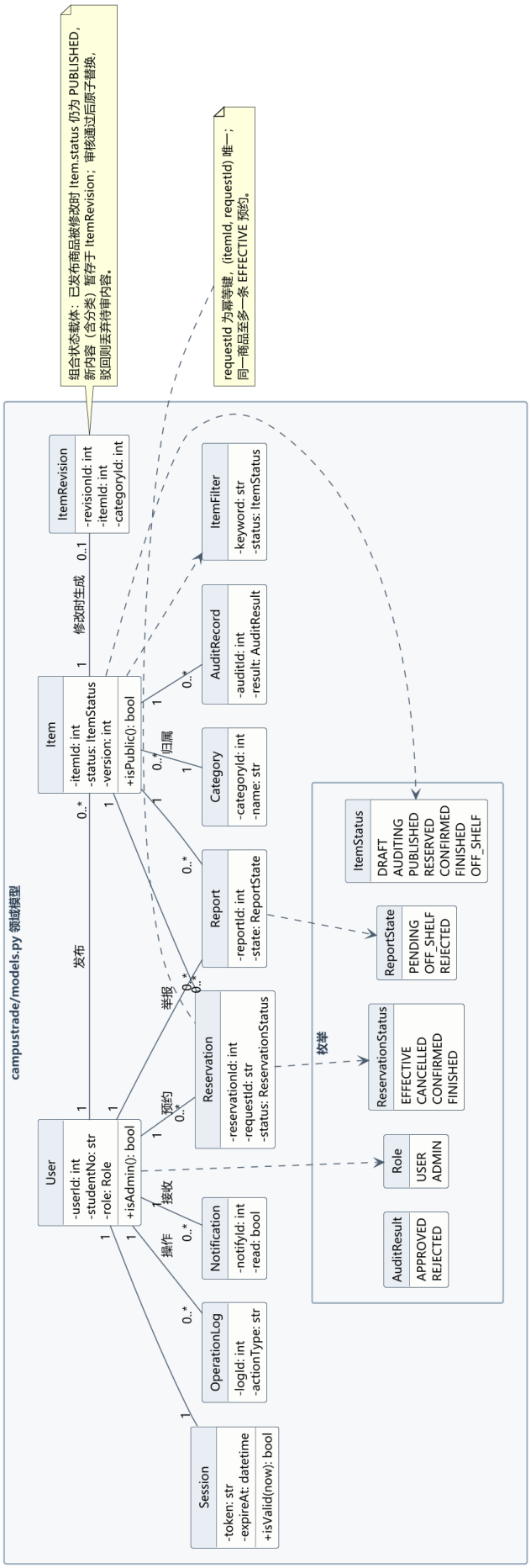


图 2-1 领域模型类图（实现版，源码见 UML/UML 类图.puml）

2.3 对实验二类图的三处修订

实现过程中发现原设计有三处需要改，改动都体现在本报告附带的类图里。

第一处：实体上的业务方法上移到服务层。实验二类图给 `Item` 挂了 `submitAudit()`、`modify()`、`offShelf()`，给 `Reservation` 挂了 `cancel()`、`confirm()`、`finish()`。真正写的时候发现这些方法都需要读别的聚合（送审要看分类是否有效、取消预约要同时改商品状态），实体会被迫持有仓储引用，事务边界也就散了。改成”实体只描述数据与自身判断、服务负责跨实体协作”之后，`Item.submitAudit()` 变成 `ItemService.submit_audit()`，状态迁移仍由 `state` 模块统一校验，规则并未丢失，只是换了执行者。

第二处：`ItemRevision` 增加 `categoryId`。原设计只暂存标题、价格、图片，但需求 REQ-005 允许按分类筛选，卖家改分类同样属于”修改商品信息”，如果不在待审内容里带上分类，审核通过后分类会与内容不一致。加一个字段即可，代价是审核通过时字段替换多一行。

第三处：新增 `ServiceBase`、`Store`、`Table` 三个实现类。它们不是领域概念，是工程结构：`ServiceBase` 收敛”令牌换用户”和”落盘”两件重复事；`Store`/`Table` 提供泛型化的持久化与查询，避免每个服务各写一遍列表遍历。

2.4 状态图的落地

状态图（图 2-3）的 7 个状态、9 条合法迁移被写成 `state.py` 里的一个字典，键是当前状态、值是允许的后继状态集合。`apply_status()` 先 `ensure_transition()` 再赋值，并递增 `version`；这个版本号是审核时”重新读取状态”的实现基础（REQ-003）。

每条箭头对应的触发动作与校验位置见表 2-3。表里没有出现的组合，代码里都走 `Conflict` 分支，例如”对已预约商品确认完成””对已完成商品再次下架”。

表 2-3 状态迁移与触发动作的对应

迁移	触发方法	附加校验
草稿 → 待审核	`ItemService.submit_audit`	标题、描述、价格必填；仅本人商品
待审核 → 已发布	`AuditService.approve`	仅管理员；版本号可比对
待审核 → 草稿	`AuditService.reject`	必须填写驳回原因；写审核记录
已发布 → 已预约	`TradeService.reserve`	锁内查重；非本人商品；幂等键
已预约 → 已发布	`TradeService.cancel`	仅卖家；预约记录置为已取消
已预约 → 已确认	`TradeService.confirm`	仅卖家；必须有有效预约
已确认 → 已完成	`TradeService.finish`	仅预约买家；`auto_finish_due` 满 7 天自动完成
已发布 → 已下架	`ItemService.off_shelf`	仅本人或管理员；已预约/已确认拒绝
已下架 → 已发布	`ItemService.restore`	仅管理员，卖家不能自行恢复

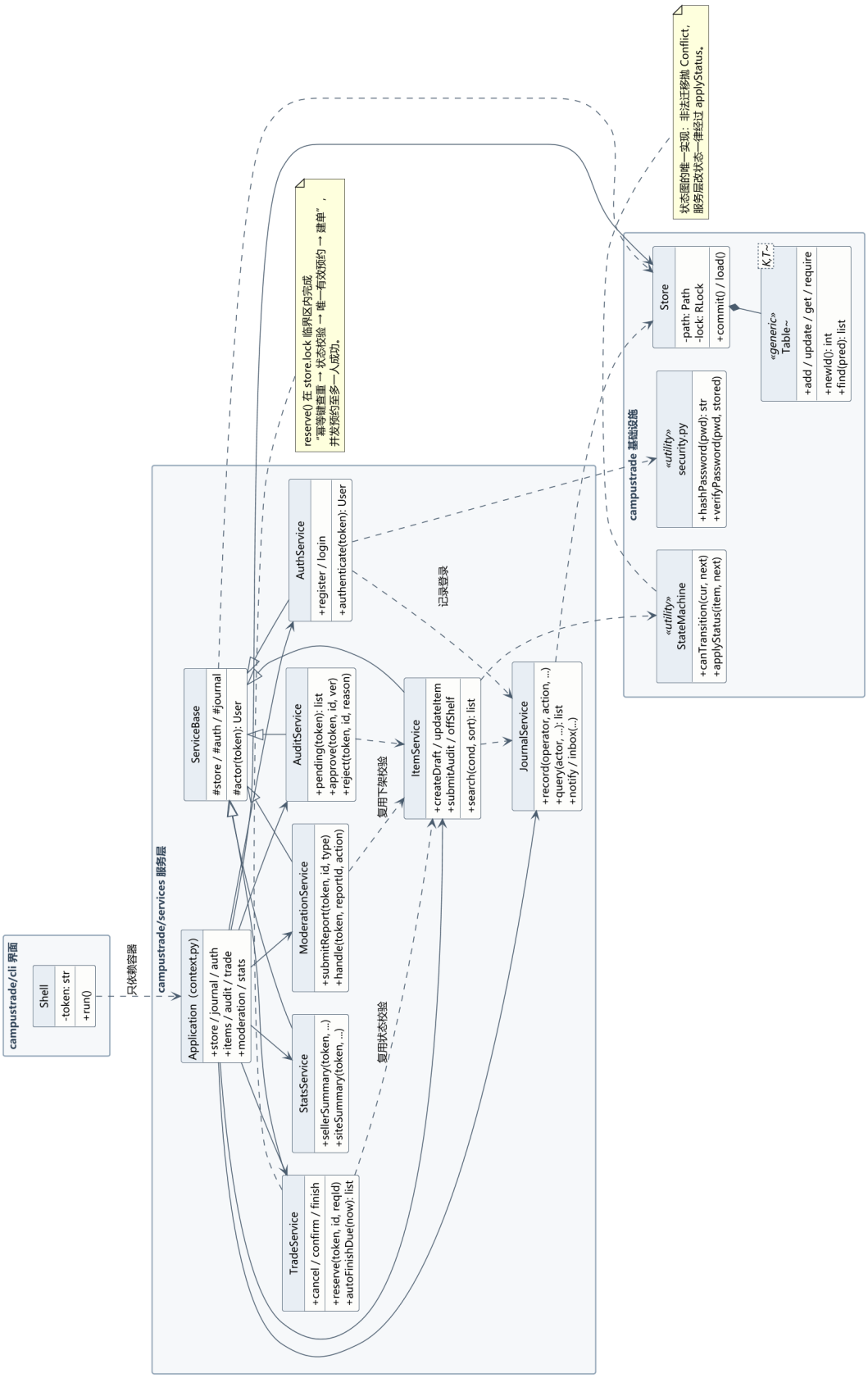


图 2-2 服务层与仓储类图（源码见 UML/UML 服务类图.puml）

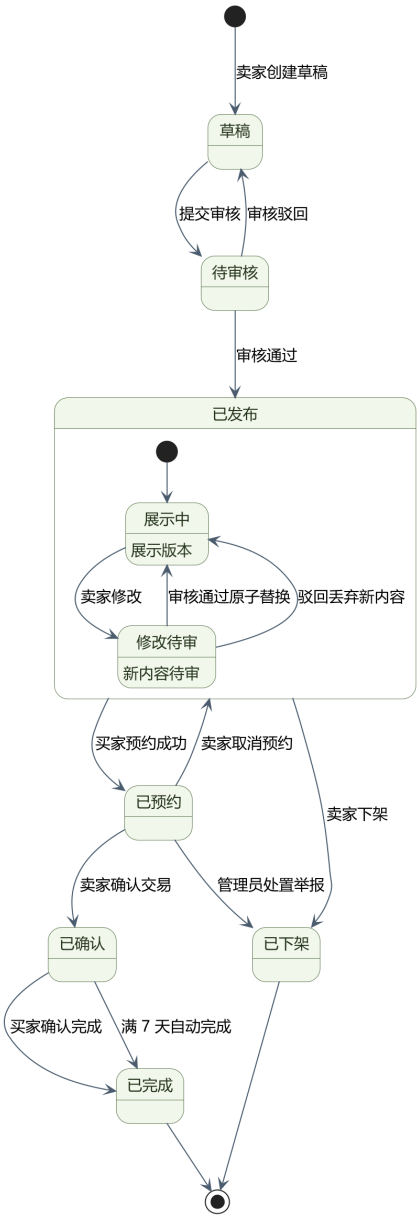


图 2-3　商品状态图（沿用实验二，实现未改动其语义）

2.5　时序图的落地

预约是最容易出错的一条流程，实验二已经画出”幂等键 + 唯一有效预约”，实现时按图 2-4逐条翻译：`alt` 的每个分支对应一个 `raise`，`with store.lock` 对应时序图上的临界区。代码里查重放在最前，因为重发的请求必须返回原结果，此时商品状态已经是”已预约”，若先校验状态就会把重发误判为冲突。

修改已发布商品是第二条关键流程（图 2-5）。它把”待审内容”与”商品状态”解耦：修改只写 `ItemRevision`，商品仍是已发布、前台仍显示旧信息，因此审核期间买家照常预约；审核通过时在一个方法里替换全部字段并删除待审记录，驳回则只删待审记录。这条流程也是实现期发现设计漏洞的地方——第 3 节案例 1。

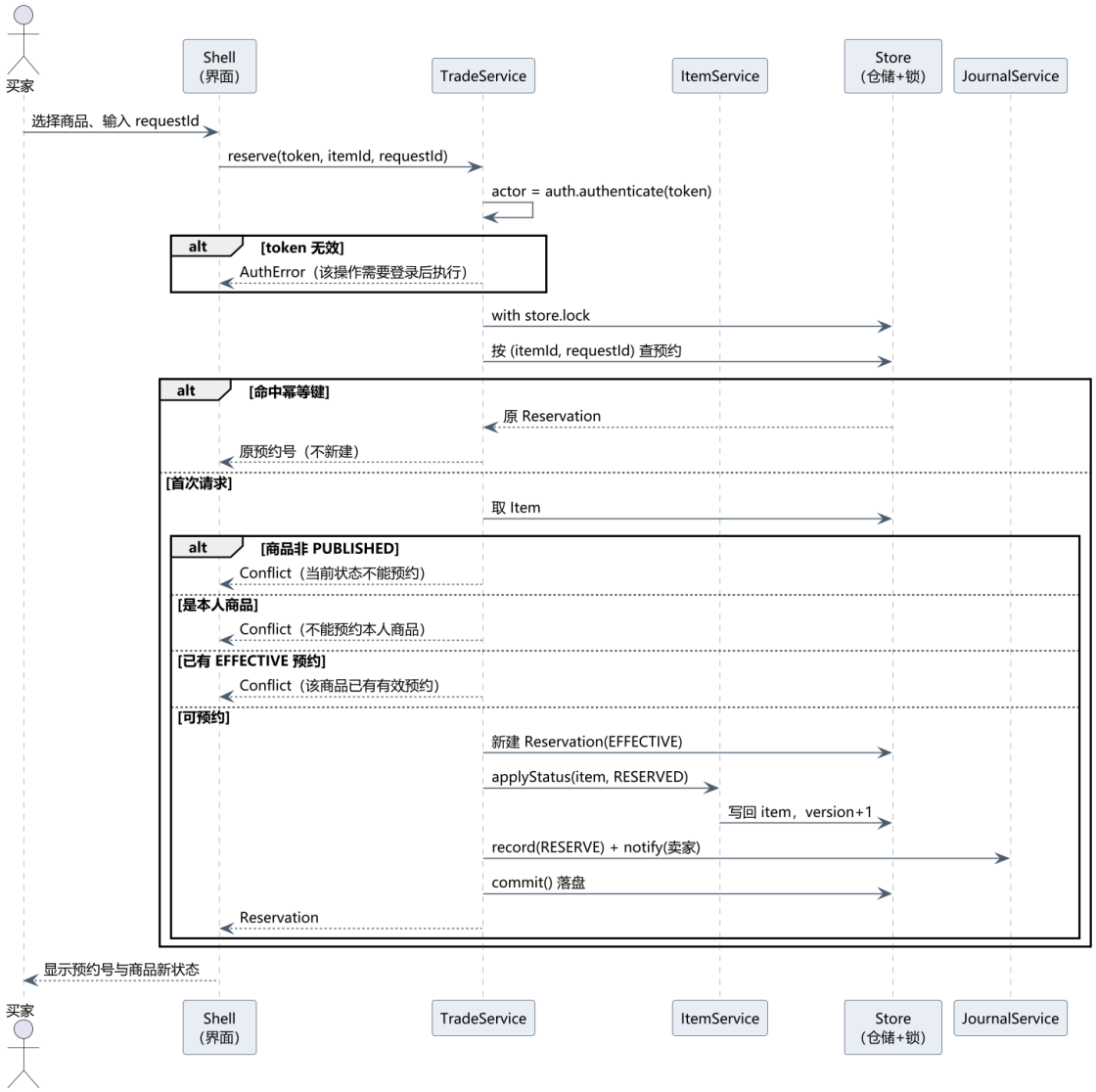


图 2-4　预约商品时序图（含幂等重发分支）

2.6　代码规模

最终交付 22 个源文件、3311 行（含空行与注释），去掉空行与整行注释后为 **2761 行**，全部为本次手写实现，没有任何 GUI 设计器生成的代码。另有 8 个测试文件、794 行、73 个用例（表 2-4）。

作为对照，实验二设计阶段的产物是 6 张 UML 图与 10 张界面稿，本次实现把它们变成了 2761 行可执行规则：其中约 1186 行（43%）是服务层的业务规则，界面与基础设施合计 754 行，剩下的是模型与装配。

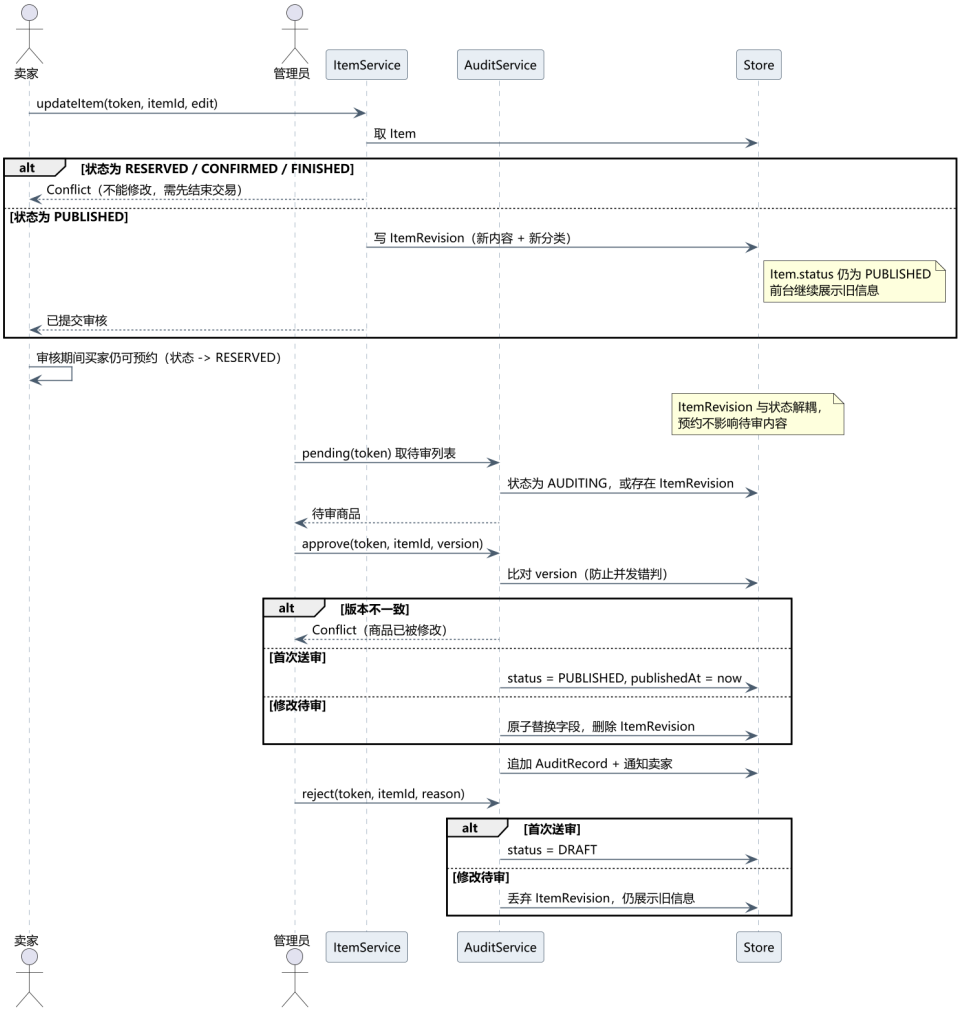


图 2-5　修改已发布商品与审核时序图

表 2-4　源代码规模统计

分组	文件数	总行数	有效代码行	主要文件
领域模型	3	513	377	`models.py`（327）
基础设施	2	305	254	`storage.py`（260）
服务层	9	1343	1186	`item.py`（386）、`trade.py`（253）
界面层	3	602	500	`shell.py`（530）
装配与入口	5	548	444	`demo.py`（344）
小计	22	3311	**2761**	平均每文件 126 行有效代码
测试	8	794	577	`tests/`（73 个用例）

3 大模型辅助开发

3.1 使用方式

直接让模型” 照着类图把系统写出来” 是我最先排除的做法：需求有 19 条、状态有 7 个，一次生成几百行代码，我既没法逐行核对，也说不清哪条规则由哪段代码保证，出错时只能整段重写。实际采用的是” 按模块切分、先规约后实现、每轮跑检查” 的方式，共十轮：

1. **一轮只喂一个模块**。输入是相关的 2–3 条需求原文、类图里对应的那一段 PlantUML、以及已经定下来的接口签名。模块小到能在一屏里核对，错了也只错这一块。
2. **先要存根与规约，再要实现**。让它先写出方法签名、参数含义、前置条件、后置条件、会抛哪些异常，我确认规约对了再让它填函数体。规约写错的概率远低于实现写错，而且在规约阶段就能发现需求理解偏差。
3. **每轮结束立刻跑三项检查**。`python -m compileall`（语法）、`pylint`（风格）、`pytest`（行为），把报错原文原样贴回下一轮，让它按报错改，而不是我手工描述错误。
4. **所有产出都过一遍手**。模型给的每一行我都读过，改动与理由写进 git 提交说明，能对得上” 哪一行是模型写的、哪一行是我改的”。

十轮的具体输入、产出与人工处理见表 3-1，完整记录（含被退回的版本）在仓库 `code/docs/prompts.md`。

表 3-1 与大模型的十轮交互摘要

轮次	输入	模型产出	人工处理
R1	19 条需求 + 类图，只做数据模型	10 个 dataclass、5 个枚举、异常层次	采纳；补 `publishedAt`、`disputed` 两个字段
R2	仓储要能落盘、支持泛型表	`Table`、`Store`、编解码	换掉 `asdict`（Decimal/datetime 会丢类型），改原子写
R3	状态图 puml	迁移表 + `apply_status`	采纳；补中文状态名映射
R4	REQ-002/005/009/014/017	商品服务	**退回**：初稿直接改已发布商品的字段，改为写待审记录
R5	REQ-003/019	审核服务与审核记录	采纳；补版本号比对
R6	REQ-001/006/007/008/015	交易服务	**退回**：初稿无锁，改为锁内查重建单
R7	REQ-010/013/018	举报、日志、统计	采纳；处置改为复用商品下架
R8	全部服务	演示脚本、命令行菜单、表格排版	采纳结构；修 3 处残留与漏传令牌
R9	pylint 初稿 27 条告警	逐条修复方案	采纳 25 条，2 条按项目约定处理
R10	pytest 失败输出	定位到” 完成只通知卖家”	采纳，改代码不改测试

3.2 大模型实际承担的工作

回头看，模型真正省时间的是三类活：

- **模板化代码**：10 个 dataclass 的字段声明、21 个文件的 `__init__`、异常类层次、`argparse` 参数定义。这些没有设计含量，但手打容易漏。
- **把图翻成表**：状态图的 9 条箭头、时序图的 `alt` 分支，让它逐条列出”箭头 → 触发方法 → 抛什么异常”，我再核对是否有漏。我在这一项上最容易看走眼，它不容易。
- **检查与修复**：把 pylint 的 27 条告警贴回去，让它给逐条修复方案（折行、改 `types.NoneType`、把多 return 改成单一出口），比我自己一条条查文档快。

而设计判断类的活它给不出可靠答案：锁加在哪里、要不要把方法从实体上移走、驳回后商品回不回草稿，这些必须我自己定。它的默认倾向是”能跑就行”，例如初稿愿意让已发布商品的修改直接覆盖字段——功能上似乎也对，但违反了 REQ-017 的展示语义。

3.3 模型生成不符合预期的四个案例

案例 1：审核服务漏掉”审核期间被预约”的状态。演示脚本执行到第 7 节时抛 `Conflict: 商品当前为已预约，不在待审核状态`。原因是模型按”审核只发生在待审核或已发布”写分支：

```
# 模型初稿：只认两种状态
if item.status is ItemStatus.AUDITING:
    apply_status(item, ItemStatus.PUBLISHED)
elif item.status is ItemStatus.PUBLISHED and revision is not None:
    self._apply_revision(item, revision, now)
else:
    raise Conflict("不在待审核状态")

# 修改后：把"可挂待审内容的状态"抽成常量，三处共用
_REVISION_HOLDERS = (ItemStatus.PUBLISHED, ItemStatus.RESERVED,
                     ItemStatus.CONFIRMED)
elif revision is not None and item.status in _REVISION_HOLDERS:
    self._apply_revision(item, revision, now)
```

REQ-017 明确写了”修改审核期间买家仍可预约”，商品会变成已预约，此时待审内容仍应能通过。这个漏洞不是笔误，是模型没有把两条需求交叉起来想。我补了 `test_revision_can_be_approved_after_reservation` 用例把它钉住。

案例 2：交易完成只通知卖家。`test_auto_finish_after_seven_days` 失败，买家收件箱里没有完成通知。REQ-015 原文是”自动将交易标记为已完成**并通知买家**”，模型把”完成”当成卖家关心的事。判断是测试对、代码错，于是改 `_finish` 同时给买卖双方发通知，测试一行没动。

案例 3：把必填校验放在建草稿时。需求要求” 草稿保存成功” 与” 必填字段缺失时不能提交” 两条同时成立，初稿在 `create_draft` 就强制描述非空，导致写了一半的草稿存不下来。改成 `_validate_fields(draft, strict)` 两档：建草稿只校验标题、价格、分类，提交审核时再查描述。

案例 4：编解码函数被 pylint 判为 `too-many-return-statements`。模型的修法是加 `# pylint: disable`，我没有接受，改成” 先解 Optional，再按分支给 `result` 赋值” 的单一出口写法。告警消失的原因是代码真的变简单了，而不是检查被关掉了。

3.4 我的判断

有三类产出我明确拒绝了：一是把业务规则塞回实体方法的版本（规则会散落，事务边界不清，见 2.3 节）；二是用 `disable` 消警告的版本；三是” 先查再写” 不加锁的并发实现（在 4 核机器上跑 4 个线程预约同一商品，初稿会建出两条有效预约）。

判断模型输出对不对，我用的是” 造反例”：每条它声称的规则，都在演示脚本或测试里造一个应该被拒绝的场景去撞它，撞不穿才算数。上面四个案例全是撞出来的，四个都不在它主动说明的风险里。这也是本次实验最大的收获——模型给的是” 看起来对” 的代码，验证的成本必须由我自己承担。

4 代码风格与静态检查

4.1 检查工具与配置

用 `pylint` 4.1.3 做静态检查，标准取 PEP 8（行宽 79、4 空格缩进），配置文件 `pylintrc` 放在代码根目录，关键项见表 4-1。只关掉了三类与作业场景无关的检查：`duplicate-code`（模板化的 dataclass 声明必然相似）、`too-few-public-methods`（实体本来就只有数据）、`too-many-lines`。设计类的上限放宽到” 参数 6 个、属性 14 个”，因为 `Item` 有 13 个字段，这是领域模型本身的样子，拆开反而更难读。

表 4-1 pylint 配置要点及理由

配置项	取值	理由
`max-line-length`	79	PEP 8 默认值，比 88/100 更严格，逼着把长表达式拆开
`max-args`	6	超过 6 个参数说明该传值对象（`ItemDraft`、`ItemEdit`）
`max-attributes`	14	领域实体字段多，但不允许无限膨胀
`disable`	4 项	见上文，均与” 模板化代码必然相似” 有关

4.2 初稿的检查结果

第一版可运行代码（不含测试）跑出 **9.85/10**，27 条告警，分布见表 4-2。值得注意的是没有一条是逻辑错误类的（`E` 级）告警，全部是格式与写法类，说明” 能跑”

和” 好读” 确实是两件事。

表 4-2　初稿 27 条告警的分布

告警	条数	典型位置	处理方式
`line-too-long` C0301	22	6 个文件，最长 83 字符	按语义折行，不缩排
`trailing-newlines` C0305	2	`item.py`、`moderation.py`	删除文件末尾多余空行
`unidiomatic-typecheck` C0123	1	`storage.py:60`	`type(None)` 改为 `types.NoneType`
`too-many-return-statements` R0911	1	`storage.py:54`	重写为单一出口
`unused-import` W0611	1	`auth.py:12`	删除未使用的 `NotFound`

完整输出如下（`python -m pylint campustrade run.py`）：

```
************* Module campustrade.demo
campustrade\demo.py:159:0: C0301: Line too long (80/79) (line-too-long)
campustrade\demo.py:182:0: C0301: Line too long (82/79) (line-too-long)
campustrade\demo.py:197:0: C0301: Line too long (80/79) (line-too-long)
campustrade\demo.py:200:0: C0301: Line too long (81/79) (line-too-long)
campustrade\demo.py:209:0: C0301: Line too long (80/79) (line-too-long)
************* Module campustrade.seed
campustrade\seed.py:45:0: C0301: Line too long (81/79) (line-too-long)
************* Module campustrade.storage
campustrade\storage.py:60:55: C0123: Use isinstance() rather than type() for a typecheck. (unidiomatic-
    typecheck)
campustrade\storage.py:54:0: R0911: Too many return statements (7/6) (too-many-return-statements)
************* Module campustrade.cli.shell
campustrade\cli\shell.py:30:0: C0301: Line too long (80/79) (line-too-long)
************* Module campustrade.services.auth
campustrade\services\auth.py:156:0: C0301: Line too long (80/79) (line-too-long)
campustrade\services\auth.py:12:0: W0611: Unused NotFound imported from campustrade.errors (unused-import)
************* Module campustrade.services.item
campustrade\services\item.py:58:0: C0301: Line too long (80/79) (line-too-long)
campustrade\services\item.py:162:0: C0301: Line too long (83/79) (line-too-long)
campustrade\services\item.py:239:0: C0301: Line too long (81/79) (line-too-long)
campustrade\services\item.py:305:0: C0301: Line too long (81/79) (line-too-long)
campustrade\services\item.py:366:0: C0301: Line too long (82/79) (line-too-long)
campustrade\services\item.py:382:0: C0305: Trailing newlines (trailing-newlines)
************* Module campustrade.services.journal
campustrade\services\journal.py:70:0: C0301: Line too long (80/79) (line-too-long)
campustrade\services\journal.py:82:0: C0301: Line too long (82/79) (line-too-long)
************* Module campustrade.services.moderation
campustrade\services\moderation.py:119:0: C0305: Trailing newlines (trailing-newlines)
************* Module campustrade.services.stats
campustrade\services\stats.py:73:0: C0301: Line too long (80/79) (line-too-long)
campustrade\services\stats.py:77:0: C0301: Line too long (80/79) (line-too-long)
************* Module campustrade.services.trade
campustrade\services\trade.py:32:0: C0301: Line too long (80/79) (line-too-long)
campustrade\services\trade.py:131:0: C0301: Line too long (81/79) (line-too-long)
campustrade\services\trade.py:133:0: C0301: Line too long (80/79) (line-too-long)
campustrade\services\trade.py:171:0: C0301: Line too long (80/79) (line-too-long)
campustrade\services\trade.py:231:0: C0301: Line too long (80/79) (line-too-long)
Your code has been rated at 9.85/10 (previous run: 9.85/10, +0.00)
```

4.3 两类典型修复

行宽。22 条里有 15 条只超 1–4 个字符，直接删空格也能过，但那样只是把问题藏起来。按语义折行才是对的：函数调用的参数对齐到左括号后，`if` 的条件用括号包起来再按布尔项分行。

```
# 修复前（83 字符）
self._journal.record(actor.user_id, action, item.item_id, True, item.title)

# 修复后：按"调用参数对齐"折行
self._journal.record(
    actor.user_id, action, item.item_id, True, item.title)
```

编解码函数的多出口。`storage.py` 的 `_decode_value` 原先用 7 个 `return` 按类型分支，pylint 报 `too-many-return-statements`。模型的建议是加 `disable`，我没有接受，改成先解 Optional、再统一给 `result` 赋值：

```
# 修复前：每个分支一个 return，7 个出口
if origin in (types.UnionType,):
    candidates = [arg for arg in get_args(hint) if arg is not type(None)]
    return _decode_value(candidates[0], raw)   # 同时触发 C0123
if origin is list:
    return [_decode_value(get_args(hint)[0], item) for item in raw]
...

# 修复后：两个出口，顺带消掉 C0123
if origin is types.UnionType:
    hint = next(arg for arg in get_args(hint) if arg is not types.NoneType)
    origin = get_origin(hint)
result = raw
if origin is list:
    result = [_decode_value(get_args(hint)[0], item) for item in raw]
elif isinstance(hint, type) and issubclass(hint, Enum):
    result = hint(raw)
...
return result
```

4.4 复查结果

把测试代码一起纳入检查后重新跑，得分 **10.00/10**：

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

测试目录用了三处文件级 `disable`，都写了理由，不是为了刷分：（1）`redefined-outer-name`——pytest 夹具互相引用是官方写法；（2）`missing-function-docstring`——用例名本身就是文档；（3）`unused-argument`——有的夹具只负责铺数据，不会出现在断言里。除此之外没有其它豁免，源码目录一条 `disable` 都没有。

5 编译与运行结果

5.1 运行环境

表 5-1 运行与检查环境

项目	版本 / 型号	说明
操作系统	Windows 11（win32）	命令行使用 Git Bash
Python	3.13.14	仅标准库；无运行时第三方依赖
代码检查	pylint 4.1.3	PEP 8，行宽 79
单元测试	pytest 9.2.0	73 个用例
UML 渲染	PlantUML 1.2024.8 + OpenJDK 17	输出 190 dpi PNG
报告排版	TeX Live 2025（XeLaTeX + ctex）	——

5.2 编译检查

Python 是解释型语言，没有独立的编译步骤；为满足” 使用编译器或 IDE 编译” 的要求，用标准库 `compileall` 对全部源码做字节码编译，它会同时暴露语法错误与模块导入错误：

```
$ python -m compileall -q campustrade run.py tests
$ echo $?
0
```

没有任何输出、退出码为 0，说明 22 个源文件与 8 个测试文件都能通过语法与导入检查。运行时验证由 `python run.py --demo`（端到端）与交互式 `python run.py --seed` 两条路径覆盖，下面分别给出真实输出。

5.3 单元测试

```
$ python -m pytest -q
```

```
.....................................................................  [ 98%]
.                                                                      [100%]
73 passed in 15.82s
```

测试按模块分文件，覆盖关系见表 5-2。这里刻意没有追求行覆盖率数字，而是每写一条规则就配一个” 应该被拒绝” 的用例：权限越权、状态不允许、重复提交、时间未到，这些分支才是真实缺陷的藏身处。

表 5-2 测试文件与覆盖的需求

测试文件	用例数	覆盖内容
test_auth.py	12	注册、登录、重复学号、弱口令、会话过期、停用账号、改密码（REQ-004/012/016）
test_item.py	15	草稿、公开查询过滤与排序、必填校验、下架与恢复、分类唯一性与引用保护（REQ-002/005/009/014）
test_audit.py	10	通过/驳回、待审内容原子替换、驳回丢弃、版本冲突、审核历史与可见性（REQ-003/017/019）
test_trade.py	13	幂等重发、唯一有效预约、取消、确认、完成、超时自动完成、异议暂停、并发竞争（REQ-001/006/007/008/015）
test_moderation.py	8	举报去重、处置下架/驳回、重复处理、已预约商品不能下架（REQ-010/014）
test_storage.py	10	JSON 快照往返（Decimal/枚举/时间）、时间范围统计、日志权限、损坏文件处理（REQ-013/018）
test_cli.py	5	登录、搜索、预约、非法选项、管理员菜单的界面冒烟
合计	73	——

5.4 端到端演示

`python run.py --demo` 在内存库上按需求编号跑 11 节场景，每节既有”应当成功”的路径，也有”应当被拒绝”的反例，最后打印是否出现未捕获异常。完整输出如下（√ 表示按预期成功，× 表示按预期被拒绝）：

```
=== 1. 注册与登录（REQ-004） ================================
  × 重复学号注册 -> 已拒绝：学号 241220101 已被注册
  × 密码错误登录 -> 已拒绝：学号或密码错误
  √ 登录态有效：241220101，角色 USER
  × 未登录直接预约 -> 已拒绝：该操作需要登录后执行

=== 2. 发布与审核（REQ-002/003/019） ========================
  √ 保存草稿 #1，状态 草稿
  √ 提交审核后状态 待审核
  × 卖家自行审核通过 -> 已拒绝：该操作仅限管理员
  √ 管理员通过后状态 已发布
  × 描述为空时提交审核 -> 已拒绝：必填字段未填写：描述
  √ 补齐描述后提交成功，状态 待审核
  × 驳回不填原因 -> 已拒绝：驳回必须填写原因
  √ 驳回后状态 草稿
  √ 审核历史 1 条，最新结论 REJECTED：缺少实拍图

=== 3. 公开查询（REQ-005） ==================================
  √ 关键词“同济”命中 1 件
      #1 线性代数 同济第七版 / 25.00 元
  √ 价格不高于 50 元命中 1 件
  × 非法状态参数 -> 已拒绝：无效的商品状态：已删除
  × 价格区间倒置 -> 已拒绝：价格下限不能高于上限

=== 4. 预约的幂等与唯一性（REQ-001） ========================
  √ 买家预约成功，预约号 1，商品状态 已预约
```

```
  √ 同一 requestId 重发返回原记录：预约号 1（共 1 条预约记录）
  × 另一买家再预约 -> 已拒绝：商品当前为已预约，不能预约
  × 卖家预约本人商品 -> 已拒绝：不能预约本人发布的商品
  × 缺少 requestId -> 已拒绝：缺少幂等键 requestId

=== 5. 取消预约（REQ-008） ================================
  × 非卖家取消预约 -> 已拒绝：只有卖家可以取消预约
  √ 卖家取消后状态 已发布
  √ 买家重新预约成功，预约号 2

=== 6. 确认交易与确认完成（REQ-006/007） ==================
  × 买家确认交易 -> 已拒绝：只有卖家可以确认交易
  √ 卖家确认后状态 已确认
  × 非预约买家确认完成 -> 已拒绝：只有预约买家可以确认完成
  √ 买家确认完成后状态 已完成
  × 重复确认完成 -> 已拒绝：商品当前为已完成，不能确认完成
  × 对已完成商品预约 -> 已拒绝：商品当前为已完成，不能预约

=== 7. 修改已发布商品（REQ-017） ==========================
  √ 修改已发布商品后，前台仍显示《罗技 M590 静音鼠标》/89.00 元，状态 已发布
  √ 新内容暂存于待审记录 #1，提交时间 21:28:23
  √ 该商品进入待审核列表：True
  √ 修改审核期间买家仍可预约：预约号 3，前台仍是旧标题《罗技 M590 静音鼠标》
  √ 审核通过后原子更新为 79.00 元，待审记录已清除：True
  √ 驳回修改后仍为已发布，显示原价 45.00 元，待审内容已丢弃：True

=== 8. 举报与下架（REQ-010/014） ==========================
  √ 举报已提交，编号 1
  √ 同一用户重复举报返回原记录：1
  × 普通用户处理举报 -> 已拒绝：该操作仅限管理员
  √ 下架后状态 已下架
  × 预约已下架商品 -> 已拒绝：商品当前为已下架，不能预约
  × 重复处理同一举报 -> 已拒绝：该举报已处理，不能重复处理

=== 9. 并发预约（REQ-001 竞争场景） ========================
  √ 4 个买家同时预约，结果：['成功', '失败（CONFLICT）', '失败（CONFLICT）',
    '失败（CONFLICT）']
  √ 商品最终状态 已预约，有效预约 1 条

=== 10. 超时自动完成（REQ-015） ============================
  √ 把确认时间回拨 8 天，模拟时间流逝
  √ 自动完成 1 笔，商品状态 已完成
  √ 卖家共收到 16 条通知，最新一条：《护眼台灯》交易已完成，金额 35.00 元

=== 11. 统计（REQ-018）与操作日志（REQ-013） ================
  √ 卖家统计：{'item_published': 6, 'item_reserved': 5, 'item_finished': 2,
    'trade_amount': Decimal('60.00')}
  √ 全站统计：{'item_total': 6, 'item_online': 0, 'user_total': 8,
    'trade_finished': 2, 'report_handled': 1, 'report_pending': 0}
  × 普通用户查全站统计 -> 已拒绝：该操作仅限管理员
  √ 操作日志共 57 条，最新一条 AUTO_FINISH / reservation_id=5
  × 普通用户查操作日志 -> 已拒绝：只有管理员可以查询操作日志

演示结束：全部操作均按预期完成或被正确拒绝。
```

几个值得单独说明的结果：

- 第 4 节：同一 `requestId` 重发返回预约号 1，预约记录仍只有 1 条，”重发不算新请求”落地；另一买家再预约被拒，唯一性成立。
- 第 7 节：修改已发布商品后前台仍显示《罗技 M590 静音鼠标》89 元，期间买家仍能预约（预约号 3），审核通过后价格变为 79 元且待审记录被清除；驳回的那一件仍显示原价 45 元。
- 第 9 节：4 个买家经屏障同步后同时预约，只有 1 个成功，其余 3 个收到 `CONFLICT`。商品最终只有 1 条有效预约。
- 第 10 节：把确认时间回拨 8 天后触发自动完成，商品变已完成，卖家与买家都收到通知。

5.5 交互式运行

`python run.py --seed` 进入菜单界面，下面是真实会话。为节省篇幅，省去了每条命令后重复打印的菜单与输入提示，只在 → 后保留该步的执行结果，其余原样保留：管理员审核通过一件待审商品，买家搜索并预约，卖家确认交易，买家确认完成，最后卖家查看统计。

```
已初始化演示数据：管理员 900000001，密码 ********
================================================================
                  校园二手交易系统 • 命令行版
================================================================

  当前身份：未登录（可匿名浏览）
  → 登录成功。

  当前身份：900000001（管理员）
  编号  标题                    状态    版本  待审修改
  ----  ----------------------  ------  ----  --------
     3  考研英语真题 2010-2024  待审核     2  无

  当前身份：900000001（管理员）
  → 审核通过，商品状态：已发布

  当前身份：900000001（管理员）
  当前身份：未登录（可匿名浏览）
  → 登录成功。

  当前身份：241220102（买家/卖家）
  → 命中 3 件
  编号  标题                     价格  状态    发布
  ----  ----------------------  -----  ------  -----------
     3  考研英语真题 2010-2024  45.00  已发布  10-31 21:28
     2  罗技 M590 静音鼠标      89.00  已发布  10-31 21:28
     1  高等数学 第七版 上册    18.00  已发布  10-31 21:28

  当前身份：241220102（买家/卖家）
  → 预约成功：预约号 1，请求号 req-cli-a

  当前身份：241220102（买家/卖家）
```

```
  预约号  商品  买家  状态  异议
  ------  ----  ----  ----  ----
       1     3     3  有效  否

  当前身份：241220102（买家/卖家）
  当前身份：未登录（可匿名浏览）
  → 登录成功。

  当前身份：241220101（买家/卖家）
  编号  标题                      价格  状态    发布
  ----  ----------------------  -----  ------  -----------
     3  考研英语真题 2010-2024  45.00  已预约  10-31 21:28
     2  罗技 M590 静音鼠标      89.00  已发布  10-31 21:28
     1  高等数学 第七版 上册    18.00  已发布  10-31 21:28
     4  待补图的书架            60.00  草稿    -

  当前身份：241220101（买家/卖家）
  → 已确认交易，请与买家线下交付。

  当前身份：241220101（买家/卖家）
  当前身份：未登录（可匿名浏览）
  → 登录成功。

  当前身份：241220102（买家/卖家）
  → 交易已完成。

  当前身份：241220102（买家/卖家）
  当前身份：未登录（可匿名浏览）
  → 登录成功。

  当前身份：241220101（买家/卖家）
  → 发布 4 件，预约 1 次，完成 1 笔，交易额 45.00 元

  当前身份：241220101（买家/卖家）
  当前身份：未登录（可匿名浏览）
```

会话中可以看到权限是按身份变化的：管理员登录后菜单多出 21–30 项（审核、举报、统计、日志、分类），普通用户只有 1–15 项；未登录时只能浏览商品。最后一行”发布 4 件，预约 1 次，完成 1 笔，交易额 45.00 元”与前面几步的操作完全对得上，说明统计口径没有把别人的数据算进来。

6 Git 远程代码管理

6.1 本地仓库与远程仓库

本地仓库在 `repo/`，远程托管在 GitHub，仓库地址：

```
$ git remote -v
origin  https://github.com/APY-tech/Software-Engineering.git (fetch)
origin  https://github.com/APY-tech/Software-Engineering.git (push)
```

主分支 `main`，实验一、实验二的产物都在同一仓库里，靠提交说明与目录区分，没

有另开分支——这门课的交付物是持续演进的同一个项目，分支反而增加合并成本。实验一的两次关键状态另打了标签 `lab1-ai-draft` 与 `lab1-final`，便于演示”AI 初稿 → 人工定稿” 的差别。

6.2 本次实验的提交

本轮工作拆成三次提交（另有报告与打包一次，见仓库最新记录），见表 6-1。每次提交只包含一类产物，提交说明末尾标注该次改动属于 AI 生成还是人工修订，可以逐条对上代码。

表 6-1 本次实验的提交记录

提交	文件数	内容	标注
`f0937f2`	34	22 个源文件 + 8 个测试文件 + 配置与说明	AI 生成初稿 + 人工修订
`128c4c1`	8	pylint 前后对比、测试与演示输出、代码规模、提示词记录	人工整理
`b7cc2cd`	12	类图、服务类图、两张时序图的 puml 与 png	人工修订

仓库最新的提交历史：

```
b7cc2cd 实验三 UML：按实现修订类图，新增服务类图与两张时序图（人工修订）
128c4c1 实验三 代码留档：pylint 前后对比、测试与演示输出、提示词记录（人工整理）
f0937f2 实验三 代码：19 条需求的 Python 分层实现，22 个文件（AI 生成初稿 + 人工修订）
d3ac093 实验二 报告修订：表4-1 每条记录显式给出优缺点，表4-2 编号与列宽（人工修改）
cbcb652 新增 README：仓库结构、重新生成方式与提交约定
8530f46 实验二 报告：5 章 17 页，新增 AI 工作流与 git 追踪两节（人工撰写，模型辅助扩写）
478a33c 回改实验一：REQ-017 驳回后状态、REQ-001 幂等键、REQ-009 模块归属（人工修改）
268e9fc 实验二 UI：10 个界面稿与可交互原型源文件
33e8c33 实验二 UML：六张图的 PlantUML 源码与渲染图（AI 生成初稿 + 人工修订）
3c19cd2 补充report.pdf需求分析报告
3395be6 最终版：修正REQ-005状态查询、REQ-017组合状态、REQ-009待确认，共19条需求（18已确认1待确认）
86f74d5 AI初稿：需求整理初稿，待人工审查
origin  https://github.com/APY-tech/Software-Engineering.git (fetch)
origin  https://github.com/APY-tech/Software-Engineering.git (push)
```

6.3 提交粒度的约定

上一轮实验定下的约定是” 一类产物一次提交，说明里写清 AI 生成还是人工修改”，这次沿用并补充了两条：一是**代码与它的检查记录分开提交**，这样” 改了什么代码” 和” 检查结果如何” 两条线各自可追溯；二是**报告单独提交**，避免把 3000 行代码和一份 PDF 混在一个提交里，导致 diff 无法阅读。

从提交历史能看出实验二的痕迹：`478a33c` 是设计期对实验一的回改（REQ-017 驳回后的状态），`33e8c33` 是六张 UML 图的初稿，说明里写明”AI 生成初稿 + 人工修订”。本次实验的 `f0937f2` 与它是同一种标注方式。

6.4 入库范围

`.gitignore` 里排除了三类东西：Python 的 `__pycache__/`、`.pytest_cache/`；程序运行产生的 `code/data/campustrade.json`（可用 `--seed` 重建，不占仓库体积）；报告排版的 LaTeX 中间产物（`.aux`、`.log`、`.toc`）。源码、UML 的 puml 与 png、测试、检查记录、报告源与 PDF 都入库。

7 与设计的差异、未覆盖项与反思

7.1 与实验二设计的五处差异

任务书允许” 实现与设计不一致”，但要求说明原因。本次一共有五处差异（表 7-1），其中第 3 处需要回改设计文档，其余是实现层面的调整。

表 7-1 实验二设计到实验三实现的差异

序号	设计	实现	原因
1	`Item`、`Reservation` 自带业务方法	方法上移到 `ItemService`、`TradeService`	方法需要跨聚合读写，实体会被迫持有仓储，事务边界不清（详见 2.3）
2	查询为独立组件 M4	并入商品服务	查询条件就是商品属性，拆开会造成两个服务同时读 `Item`
3	`ItemRevision` 只暂存标题、价格、图片	增加 `categoryId`	改分类也属于” 修改商品信息”，不带分类会导致审核后内容与分类不一致
4	新增 `ServiceBase`、`Store`、`Table`	三个实现类	收敛” 令牌换用户”” 落盘”” 泛型查询” 三处重复代码
5	前后端分离的 B/S 结构，用户端 H5	单机命令行界面	本次重点是规则正确性；服务层与界面层已隔离，换界面不动业务

第 3 处建议回改实验二类图，已在仓库的类图（`UML/UML 类图.puml`）中体现；第 5 处属于形态调整，实验二报告第 1 节写的” 形态为前后端分离的 B/S 结构” 在本次并未实现，这一点在下一小节作为未覆盖项明确列出。

7.2 未覆盖或简化的部分

诚实记录四条，避免把” 没做” 说成” 做了”：

1. **REQ-011（查询响应时间）未做实测。**需求约定”1 万条商品、20 并发、100 次查询中至少 95 次在 1 秒内”。当前实现是单机内存表 + 整库快照写盘，20 并发的语义与 Web 服务不同，测出来的数字不能直接对照这条非功能需求。合理的做法是在实验四的性能测试里针对 `ItemService.search()` 做基准，用生成的 1 万条数据测单次查询耗时，并说明并发模型差异。

2. **REQ-012 的 HTTPS 未实现。**桌面程序没有 HTTP 通信，这条需求在本形态下不适用；需求中” 密码加密存储、校验不还原明文” 的部分已实现（PBKDF2-HMAC-SHA256，12 万轮，随机盐，密文比对）。
3. **REQ-009 仍为待确认。**实现上按” 管理员可维护分类” 做了增删改查，并加了” 被商品引用的分类不能删除”” 分类名不能重复” 两条约束。这条需求在实验一就是待确认状态，实现只是给了一个可用的默认方案，等确认后再调整。
4. **没有图形界面。**实验二的 10 张界面稿没有落地。控制台界面能覆盖全部用例，但演示效果差，这是本次最明显的短板。

7.3 反思

第一，” 能跑” 离” 对” 还有很远。四个真实缺陷里有三个（审核期间被预约、完成只通知卖家、必填校验过早）在功能演示时都不报错，是演示脚本和测试里的” 反例” 把它们撞出来的。如果只按正常流程点一遍，这些缺陷会一直留在代码里。以后再写这类有规则的模块，我会先列” 应该被拒绝的场景清单”，再写实现。

第二，设计的价值在写代码时才显现。实验二的状态图让我在写 `state.py` 时只需要翻译 9 条箭头，不用边写边想” 已预约能不能下架”；而时序图里” 幂等键查重放在状态校验之前” 这个顺序，直接决定了重发请求不会被误判为冲突。反过来，设计里没画清楚的地方（改分类要不要一起待审）就变成了实现期的返工。

第三，静态检查应该早跑、常跑。第一次跑 pylint 是写完 3000 行之后，27 条告警里有 22 条是行宽，属于” 写的时候就该注意” 的问题。按模块跑、每轮清干净，改起来是几秒钟的事；攒到最后再改，就是一次几百行的机械劳动，还容易改错。

下一步。如果继续做，优先补三件事：给 `ItemService.search()` 做性能基准以满足 REQ-011；把命令行换成 Tkinter 界面复用实验二的界面稿；把 `Store` 换成 SQLite，用唯一索引在执行层再保证一次” 同一商品至多一条有效预约”，而不是只依赖服务层的锁。